

Resolución de problemas y algoritmos

Dra. Jessica Andrea Carballido

jessicarballido@gmail.com



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR



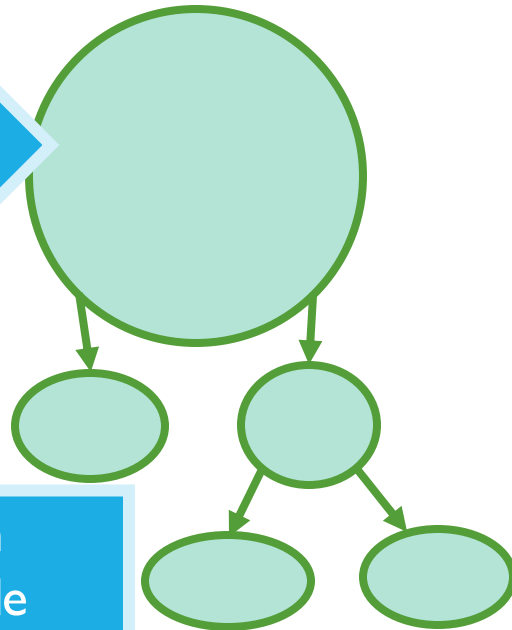
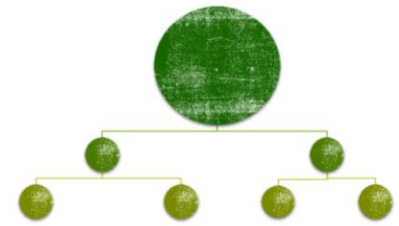
Diseño
de la solución

Dividir y conquistar

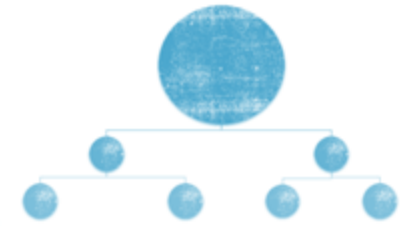
Implementación de
los sub-problemas
como funciones o
procedimientos

Implementación
de la estrategia de
resolución

Programación
modular



FUNCIONES



SIEMPRE tienen un ÚNICO dato de salida.
Ese dato de salida debe ser de tipo entero, real,
caracter o lógico.

```
function NOMBRE(.....): tipo;
```

Son
invocadas
desde
expresiones

```
function NOMBRE(datos de entrada): tipo;  
var {variables locales};  
begin  
...  
  NOMBRE:={expresión de tipo “tipo”};  
end;
```



FUNCIONES

Al **nombre de la función** se le asigna el resultado en el cuerpo de la misma.

El nombre de la función **NO ES UNA VARIABLE**.

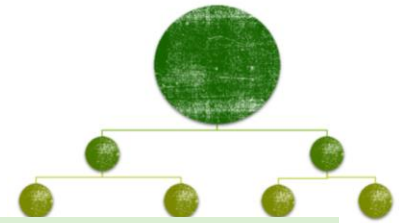
El resultado puede asignarse varias veces, quedando como **resultado final la última asignación ejecutada**.

Son invocadas desde expresiones

```
function NOMBRE(datos de entrada): tipo;  
var {variables locales};  
begin  
...  
  NOMBRE:={expresión de tipo “tipo”};  
end;
```



Funciones



Problema: A partir de una secuencia de números positivos terminada en 0, mostrar en pantalla cada número, seguido de su cantidad de dígitos.

Al final debe mostrar la cantidad total de números leídos y la máxima cantidad de dígitos encontrada.

12 2 134 21 5 0



12	2
2	1
134	3
21	2
5	1

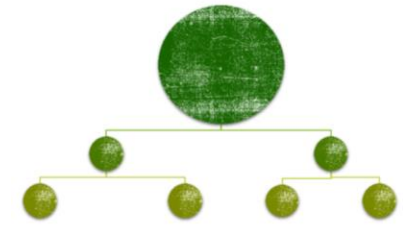
Total Nros: 5

Mayor cant digitos: 3

```
Compiling main.pas
Linking a.out
32 lines compiled, 0.1 sec
Ingrese secuencia terminada en 0
10 4 6112 7 0
Número: 10 Cantidad de dígitos: 2
Número: 4 Cantidad de dígitos: 1
Número: 6112 Cantidad de dígitos: 4
Número: 7 Cantidad de dígitos: 1
La cantidad de números es 4
La cantidad máxima de digitos es 4
```



Funciones



```
program secuenciaEnteros;  
...  
function cantDigitos (num: integer): integer;  
var contador: integer;  
Begin  
    contador := 0;  
    while (num<>0) do  
        begin  
            num := num div 10;  
            contador := contador + 1;  
        end;  
    cantDigitos := contador;  
End;  
...  
Begin  
...  
End.
```

Bottom-up



Funciones

```
begin
  writeln('Ingrese secuencia terminada en 0');
  max:=0; cantNum:=0;
  repeat
    read(n);
    if (n<>0)
    then
      begin
        cantDig:= cantDigitos(n);
        writeln(n, ' ', cantDig);
        if (cantDig > max) then
          max := cantDig;
        cantNum := cantNum + 1;

        until (n=0);
        writeln ('La cantidad de números es ', cantNum);
        writeln ('La cantidad máxima de dígitos es ', max);
      end.
end.
```

n se mantiene sin
cambios luego de
la invocación

cantDig:= cantDigitos(n);
writeln(n, ' ', cantDig);
if (cantDig > max) then
 max := cantDig;
 cantNum := cantNum + 1;

Realizo la
invocación
**UNA UNICA
VEZ**



Funciones

```
begin
```

```
...
```

```
...
```

```
writeln(n, ' ', cantDigitos(n));  
if (cantDigitos(n) > max) then  
    max := cantDigitos(n);
```

```
...
```

```
end;
```

```
writeln ('La cantidad de números es ', cantNum);  
writeln ('La cantidad máxima de dígitos es ', max);
```

```
...
```

```
end.
```

¡NO!



Si lo hiciéramos así,
tendríamos 3
invocaciones a la
función cuando se
puede resolver con
una sola.



```

1
2 program secuencia;
3 var n, cantDig, max, cantNum: integer;
4 function cantDigitos (num: integer): integer;
5 Var contador: integer;
6 Begin
7     contador := 0;
8     while (num<>0) do
9         begin
10             num := num div 10;
11             contador := contador +1;
12         end;
13     cantDigitos := contador;
14 End;
15
16 begin
17     writeln('Ingrese secuencia terminada en 0');
18     max:=0; cantNum:=0;
19     repeat
20         read(n);
21         if (n<>0)
22         then
23             begin
24                 cantDig:= cantDigitos(n);
25                 writeln('Número: ', n, ' Cantidad de dígitos: ', cantDig);
26                 if (cantDig > max) then
27                     max := cantDig;
28                 cantNum := cantNum + 1;
29             end;
30     until (n=0);
31     writeln ('La cantidad de números es ', cantNum);
32     writeln ('La cantidad máxima de digitos es ', max);
33 end.

```

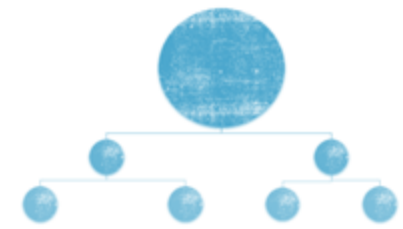
```

Compiling main.pas
Linking a.out
32 lines compiled, 0.1 sec
Ingrese secuencia terminada en 0
10 4 6112 7 0
Número: 10 Cantidad de dígitos: 2
Número: 4 Cantidad de dígitos: 1
Número: 6112 Cantidad de dígitos: 4
Número: 7 Cantidad de dígitos: 1
La cantidad de números es 4
La cantidad máxima de digitos es 4

```



Funciones



Problema: Calcular $N!$, donde N es un número natural dado.

Cuales son los datos de entrada y de salida?

Algoritmo Factorial

DE: N (natural)

DS: Factorial (natural)

Daux: i

Factorial $\leftarrow 1$

Para i desde N hasta 1

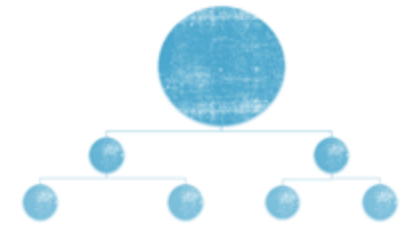
Factorial $\leftarrow$ Factorial $\times i$

¿Cuáles son los parámetros de la función?
¿De qué tipo es el resultado de la función?

```
function factorial(N: integer): integer;
```



Funciones



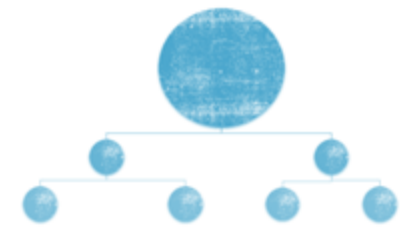
```
function factorial(N : integer): integer;  
var factAUX, i: integer;  
begin  
    factAUX := 1;  
    for i:=N downto 1 do  
        factAUX := factAUX * i;  
    factorial := factAUX;  
end;
```



Si el nombre de la función aparece en una expresión, Pascal asume que la estamos invocando. Por eso no uso “factorial” y uso “fact” en su lugar.

- La función factorial tiene una variable local **fact** y un parámetro **n**.
- La variable local **factAUX** actúa como una variable auxiliar (es como el doble de riesgo) que mantiene el valor computado y se asigna al nombre de la función (resultado), justo antes de que termine el bloque ejecutable.

Funciones



TAREA A: Mostrar por consola el factorial de cada uno de los valores naturales pares comprendidos entre a y b. El programa debe validar los valores a y b ingresados por el usuario. Validar los límites del intervalo.

Por ejemplo si $a=3$ y $b=15$ el programa debe mostrar:

Factorial de 4 es 24

Factorial de 6 es ...

Factorial de 8 es ...

...



Ejercicio tipo parcial



Funciones



Problema: Calcular m^n donde n es un número natural y m es entero.

Cuales son los datos de entrada y de salida?

Algoritmo Potencia

DE: base, exp (nat)

DS: Potencia (nat)

Daux: i

Potencia $\leftarrow 1$

Para i desde 1 hasta exp

Potencia $\leftarrow$ Potencia * base

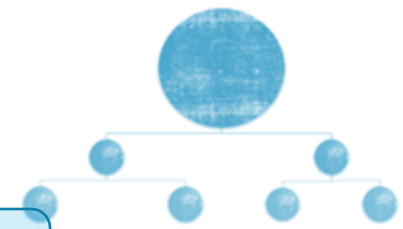
¿Cuáles son los parámetros de la función?

¿De qué tipo es el resultado de la función?

```
function potencia(base, exp: integer): integer;
```



Funciones



```
function potencia (base, exp: integer): integer;
```

```
var i, pot: integer;
```

```
begin
```

```
    pot := 1;
```

```
    for i:=1 to exp do
```

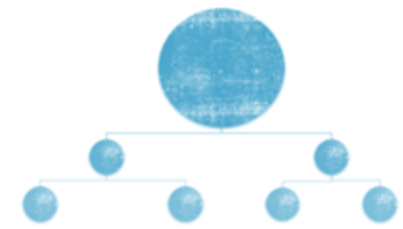
```
        pot := pot * base;
```

```
    potencia := pot;
```

```
end;
```

Encabezado

Funciones



TAREA B: Leer una secuencia de N pares de naturales (N ingresado por el usuario) y, para cada par donde el primer número sea mayor al segundo Y el segundo sea menor a 5, mostrar la potencia del primero elevado al segundo.

Por ejemplo, si la secuencia tiene 5 pares ($N = 5$), y es:

5 24 4 3 1 12 27 13 5 2

Debe mostrar:

4 elevado al 3 es 64

5 elevado al 2 es 25

Ejercicio tipo parcial





Funciones

Problema: Dado n (natural), determinar si es un número primo.
Implementarlo como primitiva.

¿Cuales son los datos de entrada y de salida?

Algoritmo esPrimo

DE: n (natural)

DS: **Es** (lógico)

Daux: Tope, posDiv (natural)

$\text{posDiv} \leftarrow 2$

$\text{Tope} \leftarrow \text{parte entera de } (\text{raiz}(n))$

Es $\leftarrow$ verdadero

Mientras (**Es** y $\text{posDiv} < \text{Tope}$) hacer

si $(n \bmod \text{posDiv} = 0)$ entonces **Es** $\leftarrow$ falso

$\text{posDiv} \leftarrow \text{posDiv} + 1$

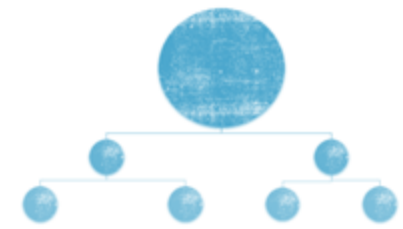
¿Cuáles son los parámetros de la función?

¿De qué tipo es el resultado de la función?

```
function esPrimo(n: integer): boolean;
```



Funciones



```
function esPrimo(n: integer): boolean;
```

```
var posDiv, Tope: integer;
```

```
    es: boolean;
```

```
begin
```

```
    posDiv := 2; Tope := trunc(sqrt(n));
```

```
    es := true;
```

```
    while (es and (posDiv < Tope)) do
```

```
        begin
```

```
            if (n mod posDiv = 0) then
```

```
                es := true;
```

```
                posDiv := posDiv + 1;
```

```
            end;
```

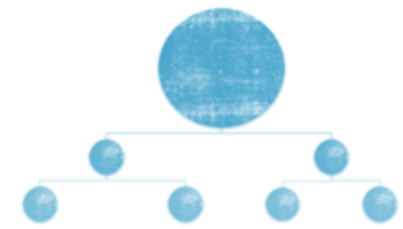
```
    esPrimo := es;
```

```
end;
```

Encabezado



Funciones



TAREA C: Dado un número N , se desea mostrar los 10 números primos posteriores a N (N podría no ser primo).

Implemente una primitiva “PrimoSig” que dado un número, devuelva el entero mayor más cercano que sea primo.

Utilice la primitiva “esPrimo” recién implementada (en la diapo anterior).

Ejercicio tipo parcial

```
Ingrese un numero:
5
Estos son los siguientes 10 numeros primos:
7  11  13  17  19  23  29  31  37  41
```



FUNCIONES

UN DATO DE SALIDA:

**Condición necesaria pero no suficiente
para implementar el sub-problema
con UNA función.**

OJO: EFECTO COLATERAL FUNCIONAL

No es recomendable...

LAS FUNCIONES:

- No deberían mostrar cosas en pantalla (write)**
- No deberían modificar el buffer (read)**
- No deben tener parámetros por variable**



Contraejemplo

Queremos escribir una primitiva que *procese una secuencia* de enteros y devuelva la suma de todos los elementos. La longitud de la secuencia es un dato de entrada a la primitiva. *El cartel pidiendo el ingreso de la secuencia y el cartel mostrando el resultado van afuera de la primitiva.*

En este caso sería aconsejable, por más que tenga un único dato de salida, implementarla como **procedimiento**.

¿Por qué? Porque tendrá que acceder al buffer con *read* (modificará al buffer, o esa, modifica algo fuera de la primitiva).

Algoritmo sumaSec

DE: long, una secuencia de enteros

DS: suma (entero)

DAUX: elemento, i

suma $\leftarrow$ 0

Para i desde 1 hasta long hacer

 leer(elemento)

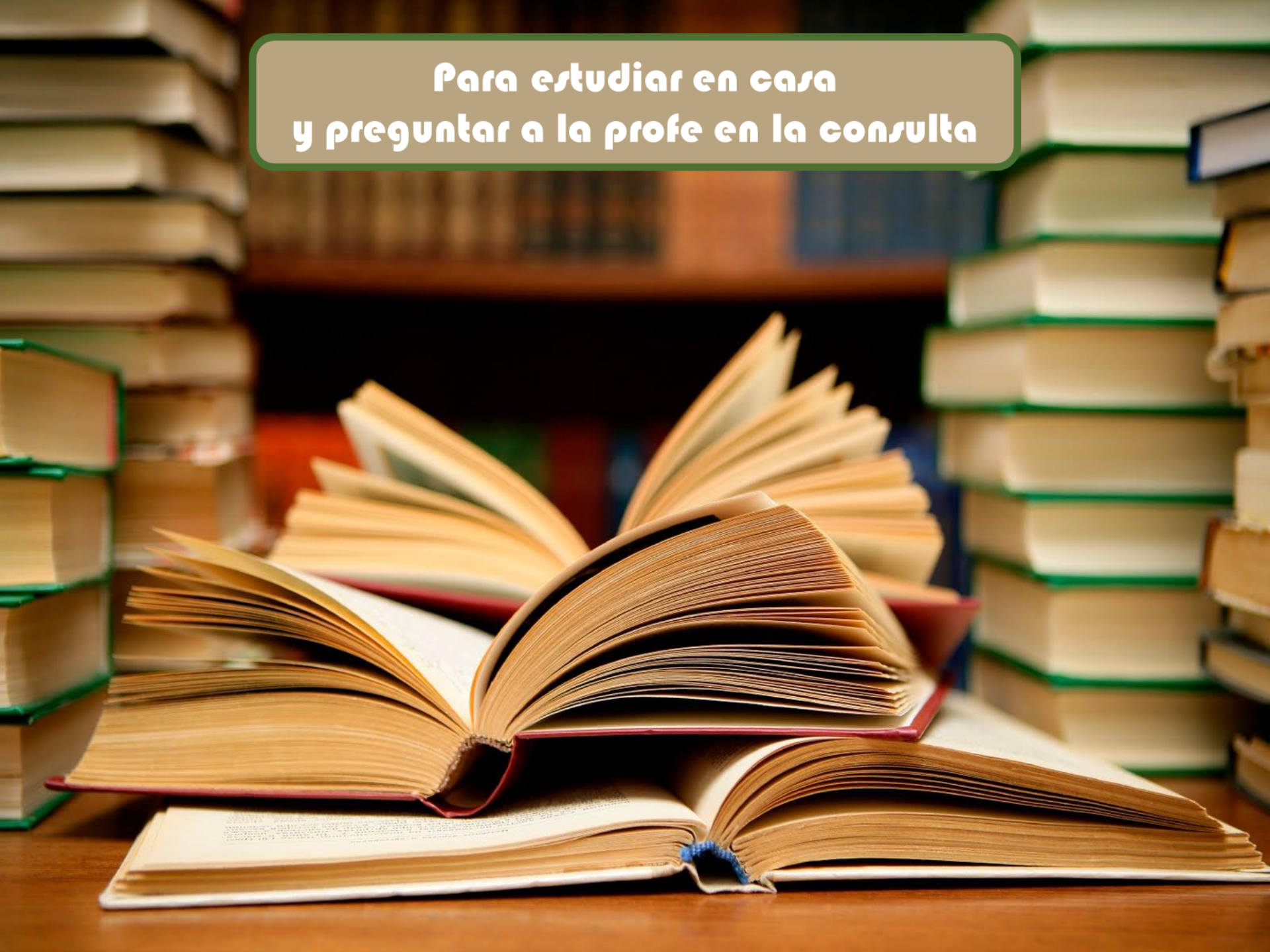
 suma $\leftarrow$ suma + elemento



```
program proced1DS;  
var LONG, S: integer;  
  
procedure sumaElementos(cantElementos: integer; var suma: integer);  
var i, unElemento: integer;  
begin  
    suma := 0;  
    for i:=1 to cantElementos do  
        begin  
            read(unElemento);  
            suma := suma + unElemento;  
        end;  
    end;  
  
begin  
    writeln('Cuantos elementos tendrá la secuencia?');  
    readln(LONG);  
    writeln('Ingrese la secuencia: ');  
    sumaElementos(LONG, S);  
    writeln('La suma de los elementos es: ', S);  
    readln;  
end.
```



**Para estudiar en casa
y preguntar a la profe en la consulta**



IMPLEMENTACION DE VARIAS PRIMITIVAS como FUNCIONES O PROCEDIMIENTOS





Problemas:

- 1) A partir de una secuencia de enteros ingresada por teclado de longitud L , mostrar para cada uno de los enteros su cantidad de dígitos.
- 2) A partir de un número $M > 0$, mostrar en distintas líneas cada dígito de M la cantidad de veces que el dígito lo indica.
- 3) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, contar cuántos enteros de la secuencia contienen al dígito D .
- 4) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, determinar si algún entero de la secuencia contiene al dígito D .



Program P1;

var L, i, N, cantDig: integer;



Definir la función cantDig

Begin

writeln('Ingrese la long de la secuencia');

readln(L);

writeln('Ingrese la secuencia');

for i:= 1 to L **do**

begin

read(N);

cantDig:=0;

while (N>0) **do**

begin

cantDig:=cantDig+1;

N:=N div 10;

end;

write(cantDig, ' ');

end;

readln;

readln;

End.



for i:=1 to L **do**

begin

read(N);

write(cantDig(N), ' ');

end;

Sub-problema:

Contar la cantidad de dígitos
de un número



```
Program Piprimit;  
var L, i, N: integer;
```

```
function cantDigitos(N: Integer): integer;
```

definiciones

```
Begin
```

```
  writeln('Ingrese la long de la secuencia');
```

```
  readln(L);
```

```
  writeln('Ingrese la secuencia');
```

```
  for i:= 1 to L do
```

```
  begin
```

```
    read(N);
```

```
    writeln('La cant de digitos de ', N, ' es ', cantDigitos(N));
```

```
  end;
```

```
  readln;
```

```
  readln;
```

```
End.
```

```
Program Piprimit;  
var L, i, N: integer;
```

```
function cantDigitos(N: Integer): integer;  
var cantDig: integer;  
begin  
    cantDig:=0;  
    while (N>0) do  
        begin  
            cantDig:=cantDig+1;  
            N:=N div 10;  
        end;  
    cantDigitos:=cantDig;  
end;
```

```
Begin  
    writeln('Ingrese la long de la secuencia');  
    readln(L);  
    writeln('Ingrese la secuencia');  
    for i:= 1 to L do  
        begin  
            read(N);  
            writeln('La cant de digitos de ', N, ' es ', cantDigitos(N));  
        end;  
    readln;  
    readln;  
End.
```



```
Program P1primit;  
var L, i, N: integer;
```

```
Program P1primit;  
var L, i, N: integer;
```

```
+ function cantDigitos(N: Integer): integer;
```

```
...
```

```
= Begin
```

```
writeln('Ingrese la long de la secuencia');
```

```
readln(L);
```

```
writeln('Ingrese la secuencia');
```

```
for i:= 1 to L do
```

```
= begin
```

```
    read(N);
```

```
    writeln('La cant de digitos de ', N, ' es ', cantDigitos(N));
```

```
end;
```

```
readln;    readln;
```

```
End.
```

```
read(N);
```

```
writeln('La cant de digitos de ', N, ' es ', cantDigitos
```

```
nd;
```

```
ln;
```

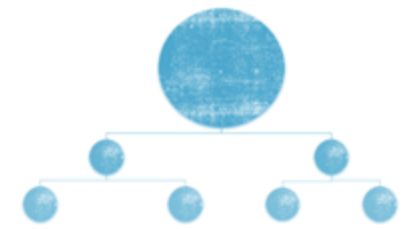
```
;
```





Problemas:

- 1) A partir de una secuencia de enteros ingresada por teclado de longitud L , mostrar para cada uno de los enteros su cantidad de dígitos.
 - 2) A partir de un número $M > 0$, mostrar en distintas líneas cada dígito de M la cantidad de veces que el dígito lo indica. 
 - 3) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, contar cuántos enteros de la secuencia contienen al dígito D .
 - 4) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, determinar si algún entero de la secuencia contiene al dígito D .
 - 5) A partir de un archivo de enteros, generar un nuevo archivo con la cantidad de dígitos de cada uno.
 - 6) A partir de un archivo de enteros, determinar si algún entero contiene al dígito D .
 - 7) A partir de un archivo de enteros, determinar si algún entero tiene todos sus dígitos pares.
- 



```
program P2;  
var M, i, digito: integer;  
begin  
  writeln('Ingrese M');  
  readln(M);  
  while (M>0) do  
  begin  
    digito:=M mod 10;  
    for i:=1 to digito do  
      write(digito);  
    writeln;  
    M:=M div 10;  
  end;  
  readln;  
  readln;  
end.
```

El objetivo de la primitiva
es mostrar algo
por pantalla!
No hay
UN DATO DE SALIDA

Sub-problema:
Mostrar el dígito d, d veces.

Visto previamente en esta clase



```

program P2;
var M, i, digito: integer;
begin
    writeln('Ingrese M');
    readln(M);
    while (M > 0) do
    begin
        digito := M mod 10;
        for i := 1 to digito do
            write(digito);
            writeln;
        M := M div 10;
    end;
    readln;
    readln;
end.

```

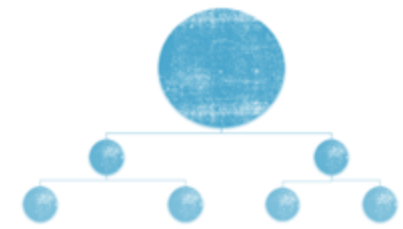


El objetivo de la primitiva
es mostrar algo
por pantalla!

Sub-problema:
Mostrar el dígito d, d veces.

No tiene datos de
salida





```
program P2primit;  
var M: integer;  
  procedure dibujarD(d: integer);  
  var i: integer;  
  begin  
    for i:=1 to d do  
      write(d, ' ');  
    writeln;  
  end;  
begin
```

Dato de entrada =
Parámetro por
valor

```
  writeln('Ingrese M');  
  readln(M);  
  while (M>0) do  
    begin  
      dibujarD(M mod 10);  
      M:=M div 10;  
    end;  
  readln;  
  readln;  
end.
```





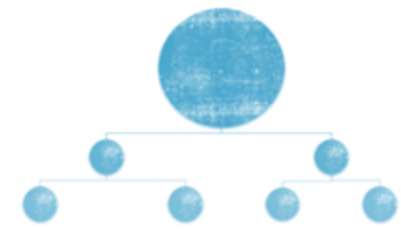
Problemas:

- 1) A partir de una secuencia de enteros ingresada por teclado de longitud L , mostrar para cada uno de los enteros su cantidad de dígitos.
 - 2) A partir de un número $M > 0$, mostrar en distintas líneas cada dígito de M la cantidad de veces que el dígito lo indica.
 - 3) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, contar cuántos enteros de la secuencia contienen al dígito D .
 - 4) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, determinar si algún entero de la secuencia contiene al dígito D .
- 
- 

```

program P3;
var N, d, cont: integer;
    fin, esta: boolean;
begin
    // cuenta la cantidad de veces que aparece
    // un dígito en cada elem de la secuencia
    writeln('Ingrese el dígito a buscar');
    readln(d);
    fin:=false;
    cont:=0;
    writeln('Ingrese la secuencia');
    while (fin=false) do
    begin
        read(N);
        if (N=0) then fin:=true
        else
        begin
            esta:=false;
            while ((N>0) and (esta=false)) do
            begin
                if (N mod 10 = d)
                then esta:=true
                else N:=N div 10;
            end;
            if esta then cont:=cont+1;
        end;
    end;
    writeln('El dígito aparece en ', cont, ' elems');
    readln;
    readln;
end.

```



Sub-problema:
Determinar si un dígito dado
está en un número.



```

program P3primit;
var N, dig, cont: integer;
    fin: boolean;

```

```

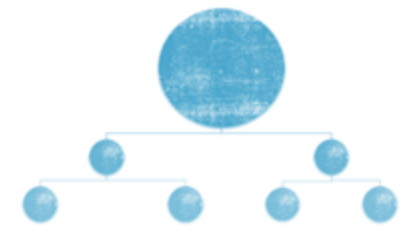
function digitoPresente(d, N: integer): boolean;
var esta: boolean;
begin
    esta:=false;
    while ((esta=false) and (N>0)) do
        if (N mod 10=d) then esta:=true
        else N:=N div 10;
    digitoPresente:=esta;
end;

```

```

begin
    writeln('Ingrese el digito a buscar');
    readln(dig);
    fin:=false;
    cont:=0;
    writeln('Ingrese la secuencia finalizando en 0');
    while (fin=false) do
        begin
            read(N);
            if (N=0)
            then
                fin:=true
            else
                if (digitoPresente(dig, N)=true)
                    then cont:=cont+1;
            end;
            writeln('El digito aparece en ', cont, ' elems');
            readln; readln;
        end.

```



Problemas:

- 1) A partir de una secuencia de enteros ingresada por teclado de longitud L , mostrar para cada uno de los enteros su **cantidad de dígitos**.
- 2) A partir de un número $M > 0$, **mostrar** en distintas líneas cada dígito de M la cantidad de veces que el dígito lo indica.
- 3) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, contar cuántos enteros de la secuencia **contienen al dígito D**.
- 4) A partir de una secuencia de enteros ingresada por teclado y que finaliza en 0, determinar si algún entero de la secuencia **contiene al dígito D**.

Implementar!



Funciones



Sucesiones

Problema: calcular el i -ésimo término de la siguiente sucesión infinita:

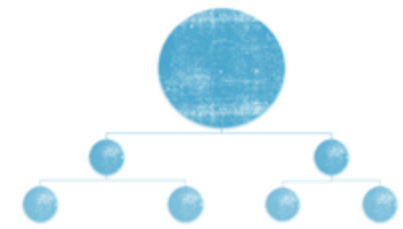
$$S = 2/1, 4/2, 8/6, 16/24, 32/120, 64/720...$$

Término general: $2^N/N!$

- Descomponer el problema en sub-problemas:
 - Calcular Potencia
 - Calcular Factorial
 - Calcular el i -ésimo Término usando *Potencia* y *Factorial*.



Funciones



ALGORITMO iEsimo

DATOS DE ENTRADA: $i \in \{\text{natural}\}$

DATOS DE SALIDA: $iEsimo \in \{\text{real}\}$

COMIENZO

$iEsimo \leftarrow \text{Potencia}(2,i)/\text{Factorial}(i)$

FIN ALGORITMO

```
function iesimo(i: integer): real;  
begin  
    iesimo := potencia(2, i) / factorial(i);  
end;
```



Si ahora quisiera escribir un algoritmo para calcular la sumatoria de los primeros K términos de la sucesión anterior...

ALGORITMO SumaKTérminos

DATOS DE ENTRADA: K {natural}

DATOS DE SALIDA: Sumatoria {real}

DATOS AUXILIARES: i {natural}

COMIENZO

Sumatoria $\leftarrow$ 0

para i desde 1 hasta K

Sumatoria $\leftarrow$ Sumatoria + iesimo(i)

Fin Repetir

FIN ALGORITMO

El dato de entrada
se lee de teclado

El dato de salida se
muestra por pantalla

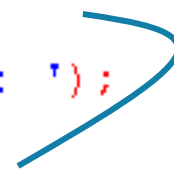
Este sería el programa principal





```
program sumaKterminos;
var i, k: integer; suma: real;
function factorial(n : integer): integer; ...
function potencia (m: real; n: integer): real; ...
function iesimo (i: integer): real;
begin
  iesimo := potencia(2, i) / factorial(i);
end;
```

```
begin
  repeat
    writeln('Cantidad de terminos a sumar: ');
    readln(k)
  until (k>=0);
  suma:=0;
  for i:=1 to k do
    suma:= suma + iesimo(i);
  writeln('La suma de los primeros ', k, ' terminos es: ', suma:2:2);
  readln;
end.
```



Validación de k



Descomposición del Problema

